

progressive assignment roadmap (~45 problems), starting from basics → image processing → ML → full pipeline.

Think of this like:

 “Build muscles before lifting the full system”

STRUCTURE

We'll divide into levels:

1. **Python + NumPy Basics**
 2. **Image Processing Fundamentals**
 3. **Feature Extraction (FFT, Noise, ELA)**
 4. **Dataset Handling**
 5. **Model Building (PyTorch)**
 6. **Training + Evaluation**
 7. **Full Pipeline Integration**
-

LEVEL 1: Python + NumPy Basics (Foundation)

👉 Goal: Be comfortable with arrays, functions, logic

♦ **Problems 1–10**

1. Write a function to normalize a list of numbers between 0 and 1
2. Implement manual matrix multiplication (no NumPy)
3. Using NumPy, create a random 3D array (image-like) and print shape
4. Convert a list into a NumPy array and reshape it to (3, 3)
5. Compute mean, std, min, max of a NumPy array
6. Normalize a NumPy array (image normalization)
7. Slice a 3D array to extract one channel
8. Stack two arrays along a new axis
9. Write a function to convert grayscale to 3-channel image
10. Clip values of an array between 0–255

🟡 LEVEL 2: Image Processing Basics (OpenCV / PIL)

👉 Goal: Understand how images are handled

◆ Problems 11–18

11. Load an image and display it
12. Resize image to 224×224
13. Convert image to grayscale
14. Normalize pixel values to [0,1]
15. Apply Gaussian blur
16. Apply edge detection (Canny)
17. Save processed image
18. Convert between PIL ↔ NumPy ↔ OpenCV formats

🟢 LEVEL 3: Feature Extraction (CORE OF YOUR PROJECT)

👉 This is where your project actually begins

◆ FFT (Frequency Domain)

19. Apply FFT to grayscale image
20. Visualize magnitude spectrum
21. Normalize FFT output to image range
22. Compare FFT of:
 - smooth image
 - noisy image

👉 Observe difference

◆ Noise Residual

23. Apply high-pass filter (kernel-based)
 24. Extract noise by subtracting blurred image
 25. Visualize noise map
 26. Compare noise:
 - real photo
 - AI-generated image
-

◆ ELA (Error Level Analysis)

27. Save image at lower JPEG quality
 28. Reload compressed image
 29. Compute difference (original - compressed)
 30. Amplify difference and visualize
 31. Compare ELA on:
 - clean image
 - edited image
-

LEVEL 4: Dataset Handling

👉 Goal: Prepare real training data

32. Load images from a folder into a list
 33. Assign labels (real = 0, AI = 1)
 34. Create a dataset dictionary structure
 35. Read dataset paths from CSV file
 36. Shuffle dataset
 37. Split into train/test (80/20)
 38. Write a custom dataset class (basic)
-

● LEVEL 5: Input Construction (VERY IMPORTANT)

👉 This is your **multi-signal fusion**

39. Generate:

- RGB
- FFT
- Noise
- ELA

for ONE image

40. Convert all to same shape (224×224)

41. Stack into multi-channel tensor

👉 Target:

(6, 224, 224)

42. Visualize each channel separately

● LEVEL 6: Model Building (PyTorch)

43. Create a simple CNN from scratch

44. Pass dummy input through model

45. Modify first layer to accept 6 channels

46. Load pretrained model (e.g., EfficientNet / ResNet)

47. Replace final classification layer

48. Add sigmoid output

LEVEL 7: Training Basics

49. Create training loop:

- forward pass
 - loss calculation
 - backward pass
-

50. Use Binary Cross Entropy loss

51. Track loss per epoch

52. Add accuracy calculation

53. Train on small dummy dataset

LEVEL 8: Data Augmentation

54. Apply:

- random resize
- flip
- blur
- compression

55. Compare model performance:

- with augmentation
 - without augmentation
-

LEVEL 9: Inference

56. Load trained model

57. Predict on single image

58. Output probability score

59. Convert score → human-readable output:

- “Likely AI”
 - “Likely Real”
-

LEVEL 10: Explainability (Bonus but powerful)

60. Implement Grad-CAM (basic)

61. Overlay heatmap on image

FINAL MINI PROJECT TASKS

62. Build full pipeline for ONE image

63. Build batch inference

64. Save predictions to CSV

65. Create simple CLI tool:

```
python predict.py image.jpg
```